

Lab 5 Challenge

Scenario

You have been asked to build a flexible order management system for a small online store.

The system must handle customers, products and orders. Different orders may contain different numbers of products and different optional information.

Your goal is to design the program using functions and the concepts covered in Lesson 5.

Unlike the standard lab, you will not be told exactly where to use every Python feature. You are expected to decide when ***args**, ****kwargs**, unpacking and different types of parameters are appropriate.

Your program should also demonstrate that you understand variable scope.

Part 1 - Products and customers

Create data for at least:

- 8 products
- 5 customers

Each product should contain information such as:

- product name
- price
- category

Each customer should contain at least:

- name
- email
- customer ID

You decide how to structure the data using concepts from previous lessons.

Part 2 - Create orders

Create a function for creating an order.

An order must contain:

- order ID
- customer
- one or more products

However, an order may also contain optional information such as:

- shipping method
- discount
- priority
- gift message
- delivery instructions

- campaign code

Your function should be flexible enough to accept different amounts of optional information without requiring every possible option to be defined beforehand.

The function should return a structured representation of the order.

Create at least **five different orders** with different products and optional information.

Part 3 - Variable number of products

Customers can purchase any number of products in one order.

Create a function that can receive a variable number of product prices and calculate the subtotal.

For example, your function should work conceptually with:

```
calculate_subtotal(199)
```

as well as:

```
calculate_subtotal(199, 349, 99, 129)
```

Do not create separate functions for different numbers of products.

Your function must also handle the situation where no prices are supplied.

Part 4 - Order configuration

Create a function that receives optional order settings. Different orders may provide different settings.

For example:

```
shipping="express"  
priority=True  
discount=10
```

while another order might provide:

```
shipping="standard"  
gift_message="Happy birthday!"
```

Your function should process the supplied settings and return them in a useful structure.

Optional settings whose value is **None** should not be included in the final result.

Part 5 - Unpacking existing data

Some customer and product information already exists in collections.

Create examples where you use existing:

- lists or tuples as positional arguments
- dictionaries as keyword arguments

Instead of manually writing every argument again, unpack the existing data when calling your functions.

Demonstrate at least:

- two examples using positional unpacking
- two examples using dictionary unpacking

At least one dictionary should contain all the information required to call one of your existing functions.

Part 6 - Flexible order summary

Create a function that produces a readable order summary.

The function should accept:

- an order ID
- a customer
- a variable number of messages or notes
- optional metadata

Example usage could conceptually look like:

```
order_summary(  
  "ORD-1042",  
  "Anna Andersson",  
  "Express delivery",  
  "Leave at reception",  
  priority=True,  
  campaign="SUMMER26"  
)
```

The function should return a readable multi-line string.

It should work even when the number of messages and optional metadata changes.

Part 7 - Scope and order statistics

Your system should keep track of some overall information, such as the store name or tax rate.

Demonstrate the difference between:

- reading a global variable from inside a function
- creating a local variable with the same name
- attempting to assign to a global numeric variable inside a function

Do not use **global** simply to make the code work.

Instead, redesign the function so that it **returns the new value**, and update the variable outside the function.

Add comments explaining why this approach is preferable.

Also create one small example demonstrating an **enclosing scope** using a nested function.

Part 8 - Order processing

Create a function that processes an order.

It should receive some required information explicitly, while also supporting:

- a variable number of products
- optional order settings

The function should calculate and return useful information such as:

- subtotal
- discount

- shipping cost
- final total

Think carefully about which parameters should be explicitly named and which information actually benefits from flexible arguments.

Do not use `*args` or `**kwargs` simply because they are available.

Your parameter design should make sense.

Part 9 - Different order types

Use your system to create and process at least:

- one order containing one product
- one order containing several products
- one order with no discount
- one order with a discount
- one express order
- one order containing additional optional metadata

Reuse the same functions wherever possible.

Avoid creating separate functions for each order type.

Final Challenge - Daily Order Report

Create a report containing information about all orders processed during the day.

The report should include at least:

- number of orders
- total revenue
- average order value
- largest order
- smallest order

Also include at least **two additional statistics or pieces of information of your own choice**.

Create a flexible report function that can receive:

- a report title
- a variable number of report sections
- optional metadata

Metadata could include:

- generated by
- department
- date
- version
- confidential status

The function should return a structured report.

Then create another function that converts the report into a readable multi-line string.

Design Challenge

Review your finished program and find at least **three places** where your choice of function parameters matters.

For each one, explain in comments:

- Why you used normal parameters, `*args` or `**kwargs`.
- What alternative design you considered.
- Why your final version is easier to use or understand.

Also identify one place where using `*args` or `**kwargs` would actually make the function **less clear** and explain why.

Requirements

Your solution must demonstrate meaningful use of:

- local and global scope
- enclosing scope
- `*args`
- `**kwargs`
- positional unpacking with `*`
- dictionary unpacking with `**`
- normal positional/named parameters
- return values
- functions calling other functions
- collections from previous lessons

Your program should:

- use descriptive function and variable names
- avoid unnecessary global state
- avoid unnecessary repetition
- separate calculations from presentation where reasonable
- run without errors

Restrictions

Do not use:

- classes
- file handling
- external libraries

These concepts belong to later lessons.

You are **not required to use `*args` or `**kwargs` in every function**. Part of the challenge is deciding when flexible parameters improve the design and when normal parameters are clearer.